

MASTER OF SOFTWARE ENGINEERING (MSE)

CAPSTONE PROJECT PROPOSAL TEMPLATE

Project Title: AI-powered Citizen Assistance System for Commune-level Administrative Procedures

1. INTRODUCTION

1.1 Background

In recent years, digital transformation has become one of the key priorities in public administration reform in Vietnam. Government agencies at all levels are actively adopting digital technologies to improve the efficiency, transparency, and accessibility of public services. At the commune level, administrative procedures such as civil registration, certification, land-related services, and social welfare support are among the most frequently requested services by citizens. These procedures often require applicants to understand complex regulations, prepare different sets of documents, and follow multiple administrative steps before submitting their applications.

Although official regulations and procedural guidelines are publicly available through government portals and legal documents, citizens still encounter significant difficulties in accessing accurate and up-to-date information. The information is typically scattered across multiple documents, written in formal legal language, and frequently updated due to changes in government regulations. Consequently, many citizens remain uncertain about the required documents, eligibility conditions, submission locations, or procedural steps, leading to repeated inquiries, incomplete applications, and unnecessary administrative workload for local government officers.

At the same time, commune-level officers spend a considerable amount of time answering repetitive questions regarding administrative procedures instead of focusing on higher-value public services. While recent advances in Large Language Models have enabled more natural and intelligent conversational systems, directly applying general-purpose AI models to administrative consultation remains challenging because these models may generate inaccurate information, lack references to official regulations, and are unable to follow structured procedural workflows.

To address these challenges, this project proposes an AI-powered Citizen Assistance System for Commune-level Administrative Procedures. The proposed system combines structured procedural workflows, knowledge management, retrieval-augmented generation (RAG), and large language models to provide accurate, context-aware, and regulation-based guidance for citizens. In addition, an administrative portal is provided to allow authorized officers to manage procedural knowledge, review extracted information from official documents, and publish updated procedure definitions, ensuring that the AI assistant always operates on verified and version-controlled administrative knowledge.

1.2 Practical problem statement and User pain-point

Despite ongoing digital transformation initiatives, citizens at the commune level continue to face considerable challenges when accessing administrative procedure information. Although official regulations and procedural documents are publicly available, the information is often distributed across multiple legal documents, written in complex administrative language, and updated frequently. As a result, citizens may struggle to determine which administrative procedure is applicable to their needs, what documents are required, whether additional conditions must be satisfied, and where or how the application should be submitted.

From the government perspective, commune officers are responsible for providing procedural guidance and answering a large number of repetitive inquiries from citizens. These consultations consume significant administrative resources, particularly for common procedures such as civil registration, certification, land-related services, and social welfare assistance. Furthermore, whenever regulations are updated, officers must manually communicate new requirements while ensuring that the information provided remains consistent with the latest official documents.

Although conversational AI and Large Language Models have significantly improved natural language understanding and response generation, directly applying general-purpose AI systems to administrative consultation presents several limitations. Such models may produce inaccurate or hallucinated responses, lack references to official regulations, and cannot reliably manage procedural workflows that require collecting mandatory information from citizens before providing appropriate guidance.

Therefore, there is a need for an intelligent assistance system that combines AI capabilities with structured procedural knowledge and workflow management. The system should not only understand citizens' natural language questions but also identify the corresponding administrative procedure, determine missing required information through guided conversations, provide regulation-based recommendations with official citations, and support administrators in maintaining an up-to-date and version-controlled knowledge base. Such a solution can improve the accessibility of public services for citizens while reducing repetitive workloads for commune-level administrative officers.

1.3 Research/Project Objectives

Overall Objective

The primary objective of this project is to design and develop an AI-powered Citizen Assistance System for Commune-level Administrative Procedures that supports citizens in accessing accurate, up-to-date, and context-aware guidance for administrative procedures through natural language interactions. The proposed system aims to improve the accessibility and efficiency of public administrative services while reducing repetitive consultation workloads for commune-level government officers.

Specific Objectives

To achieve the overall objective, the project focuses on the following specific objectives:

1. Develop an AI-powered citizen assistance portal that enables citizens to ask administrative procedure questions using natural language and receive accurate, context-aware guidance.
2. Implement a workflow-driven decision mechanism capable of identifying the appropriate administrative procedure, collecting missing required information through guided conversations, and determining when sufficient information has been gathered to provide final procedural guidance.
3. Build an administrative knowledge management portal that allows authorized officers to upload official procedural documents, review AI-generated procedure drafts, validate extracted information, and publish version-controlled procedure definitions.
4. Integrate Retrieval-Augmented Generation and Large Language Models to provide responses grounded in official administrative documents while reducing the risk of hallucinated or unsupported information.
5. Maintain version-controlled procedural knowledge so that updates to administrative regulations can be published, tracked, and applied consistently without affecting previous procedure versions.
6. Design a modular and extensible system architecture that separates business logic, AI services, knowledge management, and user interaction, enabling future extensions such as additional administrative domains, voice interaction, or integration with external government systems.

1.4 Scope and Constraints

Project Scope

The proposed system focuses on the design and development of an AI-powered citizen assistance system for commune-level administrative procedures. The implementation scope includes the following:

- Develop a **Citizen Portal** that enables citizens to obtain administrative procedure guidance through natural language conversations.
- Develop an **Admin Portal** that allows authorized officers to manage administrative knowledge, review AI-generated procedure drafts, and publish approved procedure versions.
- Support selected administrative procedures within three administrative domains at the commune level:
 - Civil Registration and Certification.
 - Land, Housing, and Planning.
 - Social Insurance and Social Welfare.
- Implement a workflow-driven conversational mechanism to identify the applicable administrative procedure, collect the required information from citizens, and provide accurate procedural guidance based on official regulations.
- Integrate Retrieval-Augmented Generation (RAG) with Large Language Models (LLMs) to generate context-aware responses grounded in verified administrative knowledge.
- Maintain version-controlled administrative procedure definitions and official document metadata to ensure that procedural guidance remains consistent with the latest approved regulations.
- Design a modular software architecture that separates user interaction, business services, AI services, and knowledge management components to facilitate system maintenance and future extensibility.

Project Constraints

The project is developed under the following constraints:

- The proposed system is designed and evaluated for **a single commune** as a proof-of-concept implementation and does not support multi-commune deployment.
- The first version supports **text-based interaction**. Speech-to-Text (STT) may be incorporated as a future extension, while Text-to-Speech (TTS) is outside the scope of this project.
- Administrative documents are assumed to be **text-based PDF files**. Automatic processing of scanned documents using Optical Character Recognition (OCR) is not included in the current implementation.
- Integration with external government platforms, such as VNeID or the National Public Service Portal, is outside the scope of this project.
- Online application submission, electronic payment, and digital signature functionalities are not considered in the proposed system.
- All AI-generated procedural knowledge must be reviewed and approved by authorized administrators before publication. The system does not automatically modify or publish official administrative procedures.
- The project is designed and implemented within the available time, resource, and academic constraints of the **Master of Software Engineering (MSE)** capstone project.

2. LITERATURE AND TECHNICAL REVIEW

2.1 Overview of existing solutions

2.1.1 Government Public Service Portals

Public service portals have become the primary digital platforms for delivering administrative services in many countries. In Vietnam, the National Public Service Portal and local government service portals provide citizens with access to administrative procedures, online public services, downloadable forms, legal documents, and application status tracking. These systems play an important role in promoting digital transformation within the public sector by centralizing administrative information and enabling citizens to access government services through web-based interfaces.

From an engineering perspective, public service portals offer several advantages. They serve as authoritative sources of administrative information, maintain official procedural documents, and provide standardized workflows for online public services. Since the published content is managed directly by government agencies, the information is generally reliable and legally valid.

However, most existing portals are designed primarily as information retrieval websites rather than intelligent conversational systems. Citizens are typically required to navigate through multiple menus, identify the appropriate administrative procedure, and manually interpret lengthy procedural documents. Users who are unfamiliar with administrative terminology or uncertain about the applicable procedure may find the process difficult and time-consuming. Furthermore, these portals generally provide static procedural descriptions and lack the capability to conduct multi-turn conversations that progressively collect missing information before providing tailored procedural guidance.

As a result, while current public service portals effectively publish official administrative information, they provide limited support for interactive and context-aware citizen assistance.

2.1.2 General-purpose Large Language Model (LLM) Chatbots

Recent advances in Large Language Models (LLMs) have significantly improved the capabilities of conversational artificial intelligence. General-purpose AI chatbots, such as ChatGPT, Google Gemini, and Microsoft Copilot, enable users to interact with AI through natural language and obtain responses across a wide range of domains. These systems demonstrate strong performance in natural language understanding, contextual reasoning, information summarization, and question answering, making them increasingly popular for knowledge-intensive tasks.

From an engineering perspective, LLM-based chatbots provide several advantages over traditional rule-based conversational systems. They can understand diverse user expressions without requiring predefined dialogue flows, support multi-turn conversations, and generate

coherent responses in natural language. Their flexibility also reduces the effort required to develop domain-specific conversational interfaces.

Despite these strengths, general-purpose LLM chatbots are designed to support broad knowledge rather than domain-specific administrative processes. They do not inherently understand the procedural workflow of government administrative services, such as identifying required information, validating mandatory conditions, or guiding citizens through sequential procedural steps. Furthermore, the responses generated by LLMs may occasionally contain factual inaccuracies or hallucinations, particularly when official regulations are unavailable or outdated within the model's knowledge. In addition, most general-purpose LLMs do not provide built-in mechanisms for version-controlled procedural knowledge, administrative approval workflows, or traceable references to official government documents.

Consequently, while LLM-based chatbots offer excellent conversational capabilities, they are insufficient as standalone solutions for delivering reliable and auditable administrative procedure guidance.

2.1.3 Workflow-based Conversational Platforms

Workflow-based conversational platforms have been widely adopted for developing task-oriented dialogue systems in domains such as customer service, banking, healthcare, and technical support. Popular platforms, including Rasa, Google Dialogflow CX, and Microsoft Bot Framework, provide structured mechanisms for intent recognition, dialogue management, slot filling, and state tracking. These capabilities enable developers to design conversational workflows that guide users through predefined sequences of interactions while collecting the information required to complete specific tasks.

From an engineering perspective, workflow-based conversational platforms offer several advantages over general-purpose LLM chatbots. Their deterministic dialogue management ensures predictable system behavior, making them suitable for applications that require controlled business logic and regulatory compliance. The use of explicit workflows also improves system reliability, simplifies testing, and enables developers to validate conversational paths before deployment. Furthermore, slot-filling mechanisms provide an effective way to collect mandatory information progressively across multiple conversation turns.

Despite these advantages, traditional workflow-based conversational platforms primarily focus on dialogue management rather than knowledge management. Administrative procedures are subject to frequent regulatory changes, requiring continuous updates to procedural rules, supporting documents, and guidance materials. Maintaining such knowledge directly within dialogue flows or rule definitions can become increasingly difficult as the number of procedures grows. In addition, these platforms generally do not provide built-in mechanisms for managing

version-controlled procedural knowledge, reviewing AI-generated administrative content, or linking responses to authoritative government documents through a structured knowledge lifecycle.

Consequently, while workflow-based conversational platforms provide robust conversational control and slot management, they are not sufficient on their own to support a maintainable, knowledge-driven administrative assistance system that requires frequent regulatory updates and traceable procedural guidance.

2.2 Technical gaps / Limitations of current approaches

Existing solutions provide valuable capabilities for digital public services and conversational AI. Government public service portals offer reliable and authoritative procedural information, general-purpose LLM chatbots provide advanced natural language understanding, and workflow-based conversational platforms enable structured dialogue management. However, none of these approaches fully satisfies the requirements of a commune-level administrative assistance system, where procedural accuracy, maintainability, regulatory compliance, and operational governance are equally important.

From an architectural perspective, existing public service portals primarily function as information publishing platforms. Although they provide official regulations and procedural guidelines, they rely heavily on manual navigation and document interpretation by citizens. These systems generally lack conversational capabilities and cannot dynamically determine the most appropriate administrative procedure or progressively collect the information required to provide situation-specific guidance.

General-purpose LLM chatbots address the limitations of traditional information retrieval by enabling natural language interaction and contextual reasoning. Nevertheless, they do not guarantee deterministic execution of administrative workflows. Their responses may vary depending on prompts and model behavior, making it difficult to ensure consistency, auditability, and regulatory compliance. Moreover, procedural knowledge embedded within foundation models cannot be updated independently of the model itself, creating challenges when administrative regulations change frequently.

Workflow-based conversational platforms improve conversation control through predefined dialogue flows and slot-filling mechanisms. However, maintaining administrative procedures directly within workflow definitions tightly couples business knowledge with conversational logic. As the number of procedures and regulatory updates increases, maintaining workflow rules becomes increasingly complex and error-prone. Furthermore, these platforms typically lack integrated mechanisms for procedural version management, document lifecycle management, and administrator-driven knowledge governance.

In addition to the individual limitations of existing approaches, current solutions rarely provide a unified architecture that combines structured procedural workflows, version-controlled administrative knowledge, retrieval-based knowledge access, and AI-assisted conversational interaction within a single maintainable platform. Consequently, government organizations often need to integrate multiple independent systems, increasing architectural complexity, maintenance costs, and operational overhead.

These technical gaps indicate the need for a solution that separates procedural knowledge from conversational logic, supports administrator-controlled knowledge management, maintains version-controlled administrative procedures, and combines deterministic workflow execution with AI-assisted natural language interaction. Such an approach forms the foundation of the system proposed in this project.

2.3 Rationale for the selected approach and stack

2.3.1 Overall System Architecture

The proposed system adopts a modular service-oriented architecture that separates user interaction, business logic, artificial intelligence, and knowledge management into independent functional components. This architectural approach was selected to improve maintainability, scalability, and separation of concerns while allowing each component to evolve independently as administrative requirements change.

Instead of embedding all functionalities within a single application, the proposed architecture distinguishes between three major responsibilities. The frontend layer is responsible for user interaction through the Citizen Portal and Admin Portal. The backend layer manages authentication, session management, workflow execution, business rules, and administrative operations. AI-related capabilities, including document processing, knowledge retrieval, slot extraction, and response generation, are encapsulated within a dedicated AI service.

This separation provides several engineering advantages. Business logic remains deterministic and independent of AI model behavior, allowing procedural workflows to be validated, tested, and audited without being tightly coupled to language model outputs. At the same time, AI capabilities can evolve independently through improvements to retrieval strategies, prompt engineering, or model replacement without requiring changes to the core business services.

Compared with a monolithic architecture that combines workflow execution and AI processing within a single application, the proposed modular architecture reduces coupling between components, simplifies maintenance, and provides greater flexibility for integrating future technologies or replacing individual services with minimal impact on the rest of the system.

Therefore, a modular service-oriented architecture provides a practical balance between

implementation complexity, maintainability, and extensibility, making it suitable for the proposed administrative assistance system.

2.3.2 Workflow-driven Conversational Architecture

A workflow-driven conversational architecture was selected as the core interaction mechanism of the proposed system to ensure deterministic execution of administrative procedures while maintaining the flexibility of natural language communication. Rather than allowing the language model to determine the entire conversation flow autonomously, the proposed architecture separates conversational reasoning from procedural decision-making.

Within the proposed system, the Large Language Model is responsible for natural language understanding tasks, such as identifying user intent, extracting relevant information from user messages, and generating fluent responses. However, all procedural decisions including identifying the applicable administrative procedure, determining required information, validating completion conditions, and selecting the next conversational action are executed by a deterministic workflow engine based on predefined procedure definitions.

This architectural separation provides several engineering advantages. First, procedural behavior becomes predictable and reproducible because business decisions are driven by structured workflow definitions rather than probabilistic model outputs. Second, workflow logic can be independently tested, validated, and audited without depending on a specific LLM provider or prompt implementation. Third, administrative procedures can evolve through updates to procedure definitions without requiring modifications to conversational prompts or retraining AI models.

Compared with architectures that rely entirely on LLM-driven reasoning, the proposed workflow-driven approach improves system consistency, regulatory compliance, and maintainability. While LLM-based autonomous agents offer greater conversational flexibility, their decision-making process is inherently non-deterministic, making it difficult to guarantee procedural correctness for government administrative services where accuracy and traceability are essential.

Therefore, the proposed architecture adopts a deterministic workflow engine as the decision authority while leveraging LLMs solely as intelligent language processing components. This separation ensures that conversational intelligence enhances, rather than replaces, officially defined administrative procedures.

2.3.3 Version-controlled Knowledge Management

Administrative procedures are subject to continuous changes due to updates in government

regulations, legal documents, and local administrative policies. Consequently, maintaining procedural knowledge as static application logic or hard-coded conversational rules introduces significant maintenance challenges and increases the risk of providing outdated guidance to citizens. To address this issue, the proposed system adopts a version-controlled knowledge management approach in which procedural definitions are managed as structured data rather than embedded within application code.

In the proposed architecture, each administrative procedure is represented as a versioned procedure definition that specifies the procedural workflow, required information, conversational rules, guidance content, and references to official documents. New regulations are incorporated by publishing new procedure versions while preserving previous versions for traceability and historical reference. This approach enables administrative knowledge to evolve independently of the application software, reducing deployment frequency and simplifying regulatory updates.

From an engineering perspective, separating procedural knowledge from application logic provides several advantages. Administrative officers can review and approve updated procedure definitions through the administration portal without requiring software modifications. Version-controlled definitions also support rollback mechanisms, allowing previous versions to be restored if newly published procedures contain errors or inconsistencies. Furthermore, maintaining explicit version histories improves auditability by enabling the system to identify which procedural version was used when generating guidance for a particular citizen interaction.

Compared with systems that directly encode procedural rules within application code or conversational workflows, the proposed knowledge management approach significantly improves maintainability and governance. Procedural updates become data management operations rather than software development tasks, allowing regulatory changes to be incorporated more rapidly while preserving system stability.

Therefore, version-controlled knowledge management provides a reliable foundation for maintaining administrative procedures in environments where regulations change frequently and procedural correctness must be preserved over time.

2.3.4 Retrieval-Augmented Generation (RAG)

The proposed system adopts a Retrieval-Augmented Generation (RAG) approach to enhance the reliability and maintainability of AI-generated responses. Instead of relying solely on the internal knowledge of a Large Language Model, the system retrieves relevant information from an administrator-managed knowledge base before generating responses. This approach ensures that conversational outputs are grounded in official administrative documents and approved procedural definitions.

In the proposed architecture, administrative documents are processed through the knowledge

management pipeline, where relevant content is extracted, structured, and indexed for retrieval. During citizen interactions, the system first determines the applicable administrative procedure and retrieves the corresponding procedural definition. When additional regulatory information or explanatory content is required, relevant knowledge is retrieved from the indexed knowledge base and incorporated into the response generation process. Consequently, the language model generates answers based on verified contextual information rather than relying exclusively on its pretrained knowledge.

From an engineering perspective, RAG offers several advantages over directly embedding administrative knowledge into LLM prompts or relying on model pretraining alone. Official regulations frequently change, while retraining or fine-tuning foundation models is computationally expensive and operationally impractical for local government organizations. By externalizing procedural knowledge into a managed knowledge repository, updates can be incorporated simply by publishing new administrative documents and refreshing the retrieval index, without modifying the application or retraining the language model.

Furthermore, the proposed RAG architecture improves transparency by enabling responses to reference authoritative administrative documents. Since retrieved knowledge originates from administrator-approved sources, the system can provide traceable procedural guidance while reducing the likelihood of unsupported or hallucinated responses. This capability is particularly important in administrative domains where the accuracy and provenance of information are critical.

Compared with standalone LLM-based systems, Retrieval-Augmented Generation provides a practical balance between conversational intelligence, knowledge maintainability, and regulatory reliability. It enables the proposed system to continuously evolve with changing administrative regulations while maintaining consistent and trustworthy guidance for citizens.

2.3.5 Technology Stack Selection

2.3.5.1 Frontend Framework

The frontend of the proposed system is implemented using React with JavaScript. The selection was primarily motivated by the need to develop two independent web interfaces a Citizen Portal for conversational interactions and an Admin Portal for procedural knowledge management while maintaining reusable user interface components and efficient client-side state management.

Several modern frontend frameworks, including React, Angular, and Vue.js, were considered during the technology selection process. Angular provides a comprehensive enterprise framework with strong architectural conventions; however, its steeper learning curve and relatively heavier project structure introduce additional complexity for a medium-scale academic

project. Vue.js offers simplicity and ease of adoption but has comparatively smaller enterprise adoption and ecosystem support for large-scale applications.

React provides a practical balance between flexibility, component reusability, ecosystem maturity, and long-term maintainability. Its component-based architecture enables reusable interface elements across both portals while supporting incremental feature development without imposing rigid architectural constraints. Furthermore, React integrates well with RESTful APIs and modern frontend tooling, making it suitable for the modular architecture proposed in this project.

Considering the project scope, development resources, and maintainability requirements, React provides an appropriate engineering trade-off between development efficiency and architectural flexibility.

2.3.5.2 Backend Service

The backend of the proposed system is implemented using the Go programming language with the Gin web framework. This decision was driven by the system's primary responsibility of managing business workflows, session state, authentication, administrative operations, and communication between the frontend, AI services, and data layer.

Several backend technologies, including Java Spring Boot, Node.js, and Go, were considered during the architectural design process. Spring Boot provides a comprehensive enterprise ecosystem with mature tooling, dependency injection, and extensive framework support. However, its larger runtime footprint and relatively higher implementation complexity make it less suitable for a lightweight service-oriented architecture where most business logic consists of deterministic workflow execution rather than complex enterprise integrations. Node.js offers rapid development and an event-driven programming model, but its single-threaded execution model and dynamically typed ecosystem require additional engineering effort to maintain long-term reliability in business-critical services.

Go provides a practical balance between development simplicity, runtime performance, and operational efficiency. Its lightweight concurrency model, fast startup time, and low memory consumption make it well suited for stateless RESTful services that process concurrent citizen requests. Furthermore, Go's simple language design and strong standard library facilitate the implementation of deterministic business logic while reducing unnecessary architectural complexity.

Within the proposed architecture, the backend service functions as the system orchestrator. It is responsible for authentication, conversation session management, workflow execution, administrative knowledge management, and communication with external AI services. By separating deterministic business logic from AI-specific processing, the backend maintains

predictable system behavior regardless of the language model being used.

Considering the project's architectural requirements, Go provides an appropriate engineering trade-off between runtime efficiency, implementation simplicity, maintainability, and long-term extensibility.

2.3.5.3 AI Processing Service

The proposed system isolates AI-related functionalities within a dedicated Python-based AI service rather than embedding them directly into the backend application. This architectural decision was made to separate deterministic business operations from computationally intensive AI tasks, resulting in a more maintainable and extensible system architecture.

Several architectural alternatives were considered during system design. One approach was to implement all functionalities within a single backend service using Go. Although this approach simplifies deployment and reduces inter-service communication, it tightly couples business logic with AI processing, making the system more difficult to maintain as AI capabilities evolve. In addition, the Go ecosystem provides relatively limited support for modern AI frameworks, document processing libraries, and machine learning tooling compared with Python.

Python was selected because it has become the de facto standard language for artificial intelligence and natural language processing. Its mature ecosystem provides extensive support for document parsing, embedding generation, vector retrieval, prompt engineering, and integration with Large Language Models. These capabilities enable the AI service to efficiently perform document processing, procedural knowledge extraction, retrieval-augmented generation, and structured information extraction while remaining independent of the core business services.

Within the proposed architecture, the AI service is responsible exclusively for AI-oriented operations, including processing uploaded administrative documents, generating procedural drafts, retrieving relevant knowledge, extracting structured information from user conversations, and generating context-aware responses. Business decisions, workflow execution, authentication, and session management remain under the control of the backend service. This clear separation of responsibilities ensures that modifications to AI models, prompting strategies, or retrieval techniques do not affect the deterministic business workflow implemented by the backend.

Compared with a monolithic architecture that combines AI processing and business logic within a single service, the proposed design significantly improves modularity and maintainability. The AI service can evolve independently, allowing new language models, embedding techniques, or retrieval algorithms to be adopted without impacting the rest of the system. Furthermore, AI workloads can be scaled independently according to computational demand while preserving

the stability of the core administrative services.

Therefore, the separation between the backend service and the AI processing service provides a practical engineering trade-off between architectural simplicity, maintainability, extensibility, and future scalability.

2.3.5.4 Data Storage

The proposed system adopts PostgreSQL as the primary data storage platform, with pgvector integrated as an extension for vector-based knowledge retrieval. This architecture was selected to provide a unified data management platform capable of supporting both structured administrative data and semantic retrieval while minimizing infrastructure complexity.

Several database architectures were considered during system design. A document-oriented database such as MongoDB offers flexibility for storing semi-structured data, while dedicated vector databases such as Qdrant, Milvus, or Pinecone provide optimized similarity search for Retrieval-Augmented Generation (RAG). However, adopting multiple specialized databases introduces additional operational complexity, including separate deployment, synchronization, backup strategies, security policies, and maintenance processes.

The proposed system primarily manages structured and relational information, including users, conversation sessions, procedural definitions, document metadata, workflow versions, and audit logs. These entities require transactional consistency, referential integrity, and version management, making a relational database a more appropriate choice than a schema-less alternative. PostgreSQL provides mature support for these requirements while also offering extensibility through the pgvector extension, enabling vector embeddings to coexist with structured administrative data within the same database platform.

From an engineering perspective, consolidating structured and vector data within PostgreSQL simplifies the overall system architecture. Administrative documents, procedural definitions, and their corresponding embeddings remain logically connected through the same relational model, reducing data synchronization overhead between independent storage systems. This approach also simplifies backup, disaster recovery, access control, and operational monitoring by managing a single database platform rather than multiple specialized services.

Although dedicated vector databases may provide superior performance for large-scale semantic search workloads, the expected deployment scale of the proposed system is limited to commune-level administrative services, where the volume of procedural documents and knowledge chunks remains relatively modest. Under these conditions, PostgreSQL combined with pgvector provides sufficient retrieval performance while significantly reducing infrastructure complexity and operational costs.

Therefore, PostgreSQL with pgvector offers an appropriate engineering trade-off between relational data management, semantic retrieval capability, operational simplicity, and long-term maintainability for the proposed system.

2.3.5.5 AI Model Integration

The proposed system integrates Large Language Models (LLMs) through a provider abstraction layer rather than coupling the application to a specific AI vendor. This architectural decision aims to improve flexibility, maintainability, and long-term adaptability as the AI ecosystem continues to evolve rapidly.

Several integration strategies were considered during system design. One option was to directly integrate a single commercial LLM provider, such as OpenAI or Google Gemini, throughout the application. While this approach simplifies the initial implementation, it tightly couples business services and AI workflows to a specific vendor's API, making future migration, pricing adjustments, or policy changes difficult to accommodate.

Instead, the proposed architecture introduces an abstraction layer within the AI service that standardizes interactions with external language models. Core AI capabilities including document analysis, procedural draft generation, structured information extraction, and response generation are implemented through common service interfaces rather than provider-specific implementations. As a result, the underlying LLM provider can be replaced or extended with minimal impact on the rest of the system.

From an engineering perspective, this design provides several advantages. It reduces vendor lock-in, improves maintainability, and allows the system to adopt newly available language models without modifying business services or workflow logic. Furthermore, the abstraction layer enables different models to be evaluated or deployed according to practical considerations such as response quality, operational cost, latency, or regulatory requirements.

Although introducing an abstraction layer adds a small amount of implementation complexity, the long-term architectural benefits outweigh this initial overhead. AI technologies continue to evolve rapidly, and maintaining the flexibility to adopt improved models represents a significant advantage for systems expected to operate over an extended period.

Therefore, the proposed architecture treats Large Language Models as interchangeable infrastructure services rather than fixed system components, ensuring that future AI model evolution can be accommodated without affecting the overall software architecture.

2.3.5.6 Deployment and Infrastructure

The proposed system adopts a containerized deployment architecture in which the frontend application, backend API, and AI processing service are packaged and deployed as independent services. This approach was selected to preserve the separation of responsibilities established in the application architecture while simplifying environment configuration, deployment, and future system maintenance.

Docker is used to create reproducible runtime environments for the React frontend, Go backend, and Python AI service. Containerization reduces differences between development, testing, and production environments by packaging each service together with its required dependencies. It also allows individual services to be updated or restarted independently without requiring the entire system to be redeployed.

A reverse proxy, such as Nginx or Caddy, is positioned at the system entry point to manage HTTPS termination and route incoming requests. Requests for the web interface are forwarded to the frontend container, while API requests are routed to the Go backend. The Python AI service remains inaccessible directly from the public network and is invoked internally by the backend service. This arrangement reduces the exposed attack surface and ensures that authentication, authorization, and business-level validation remain under the control of the backend API.

For the initial commune-level deployment, Docker Compose is selected instead of Kubernetes. Kubernetes provides advanced orchestration capabilities, including automated scaling, self-healing, and service discovery, but it introduces substantial configuration and operational overhead. Given the expected workload and limited number of services in the proposed system, Docker Compose provides sufficient orchestration capability while remaining easier to deploy, operate, and troubleshoot. The architecture nevertheless preserves service boundaries, allowing migration to Kubernetes in a future large-scale deployment if operational requirements increase.

Administrative PDF files are stored separately from the relational database using object storage. PostgreSQL stores only document metadata, including file location, checksum, effective date, and version references. This separation prevents large binary files from increasing the relational database size and enables document storage to be scaled, backed up, or replaced independently. A self-hosted object storage service such as MinIO is suitable for the initial deployment, while cloud-based alternatives may be adopted if the deployment environment changes.

The deployment process is supported by a Continuous Integration and Continuous Deployment pipeline. Source code changes are automatically checked through linting and unit tests before container images are built. Approved images are then pushed to a container registry and deployed to a staging environment before production release. Sensitive information, including database credentials, authentication secrets, and LLM API keys, is supplied through

environment-specific secret configuration and is not stored in the source code repository.

Compared with deploying all components as a single application process, the proposed approach introduces additional networking and configuration requirements. However, this overhead is justified by improved service isolation, reproducibility, maintainability, and deployment flexibility. Therefore, containerized services, a reverse proxy, lightweight orchestration, object storage, and automated delivery provide an appropriate deployment strategy for the scope and operational requirements of the proposed system.

3. PROPOSED SOLUTION AND ENGINEERING DESIGN

3.1 High-level system architecture

The proposed system adopts a modular architecture that separates user interaction, deterministic business processing, AI capabilities, and data management into clearly defined components. The architecture supports two primary user groups: citizens seeking administrative guidance and commune officers responsible for managing procedural knowledge.

The presentation layer consists of the Citizen Portal and the Admin Portal. The Citizen Portal provides a conversational interface through which users can describe their administrative needs, answer follow-up questions, and receive procedure-specific guidance. The Admin Portal enables authorized officers to upload official documents, review AI-generated procedural drafts, manage procedure versions, and publish approved knowledge for citizen use.

The Go Backend Service acts as the central application and business coordination layer. It handles authentication, authorization, conversation sessions, workflow execution, administrative operations, and communication between the frontend applications and the AI service. The backend also contains the deterministic Decision Engine, which evaluates procedural definitions and current session data to determine whether the system should answer directly, request additional information, or provide final guidance.

AI-related operations are delegated to a dedicated Python AI Service. This service performs document processing, structured information extraction, procedural draft generation, semantic retrieval, and natural-language response generation. It communicates with external Large Language Model providers through an abstraction layer, ensuring that the system architecture does not depend on a specific AI vendor.

PostgreSQL serves as the primary data platform for structured information, including user accounts, conversation sessions, messages, procedural definitions, document metadata, workflow versions, and audit logs. The pgvector extension stores document embeddings and supports semantic retrieval for the Retrieval-Augmented Generation process. Administrative

PDF files are stored separately in S3-compatible object storage, while their metadata and references remain in PostgreSQL.

The main citizen interaction flow begins when a user submits a message through the Citizen Portal. The backend creates or retrieves the current conversation session and forwards the message to the AI service for intent identification and structured slot extraction. The Decision Engine then loads the active procedural definition, evaluates the collected information, and determines the next system action. When additional information is required, the system generates an appropriate follow-up question. Once all mandatory conditions are satisfied, the Knowledge Service retrieves relevant approved content and produces the final guidance with supporting references.

The administrative knowledge flow begins when an authorized officer uploads an official document through the Admin Portal. The document is stored in object storage and processed by the AI service to generate a draft procedural definition. The officer reviews and corrects the generated content through a structured workspace before approving publication. The approved definition is stored as a new version, indexed for semantic retrieval, and activated for future citizen conversations. Previous versions are preserved to support auditability and rollback.

This architecture ensures that AI-generated outputs do not directly control administrative workflows or become publicly available without human approval. Deterministic decisions remain within the backend, while AI components are used for language understanding, knowledge extraction, retrieval, and response generation. The separation improves maintainability, traceability, and extensibility while remaining practical for a commune-level proof-of-concept deployment.

Figure 3.1.1 Citizen Runtime Architecture

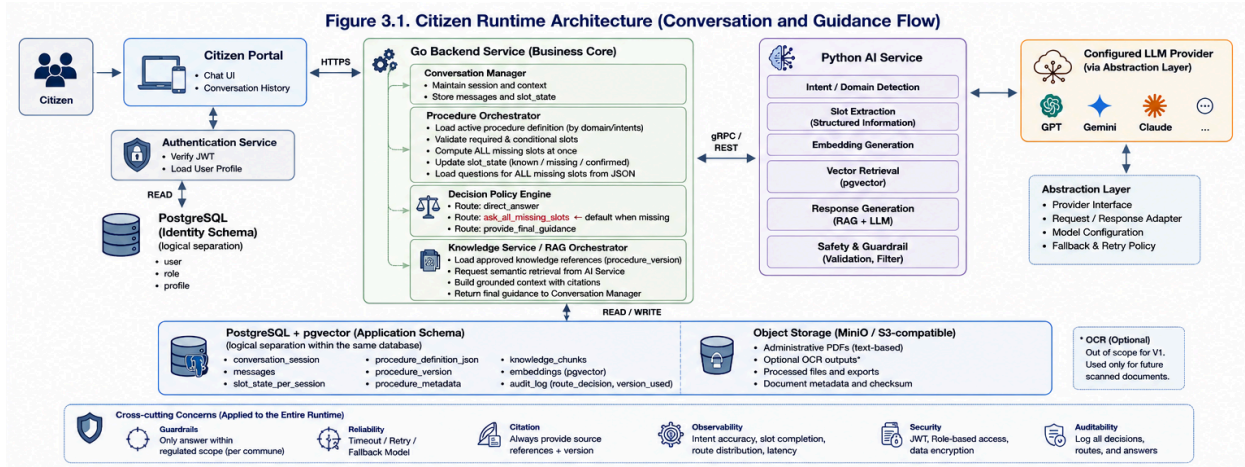
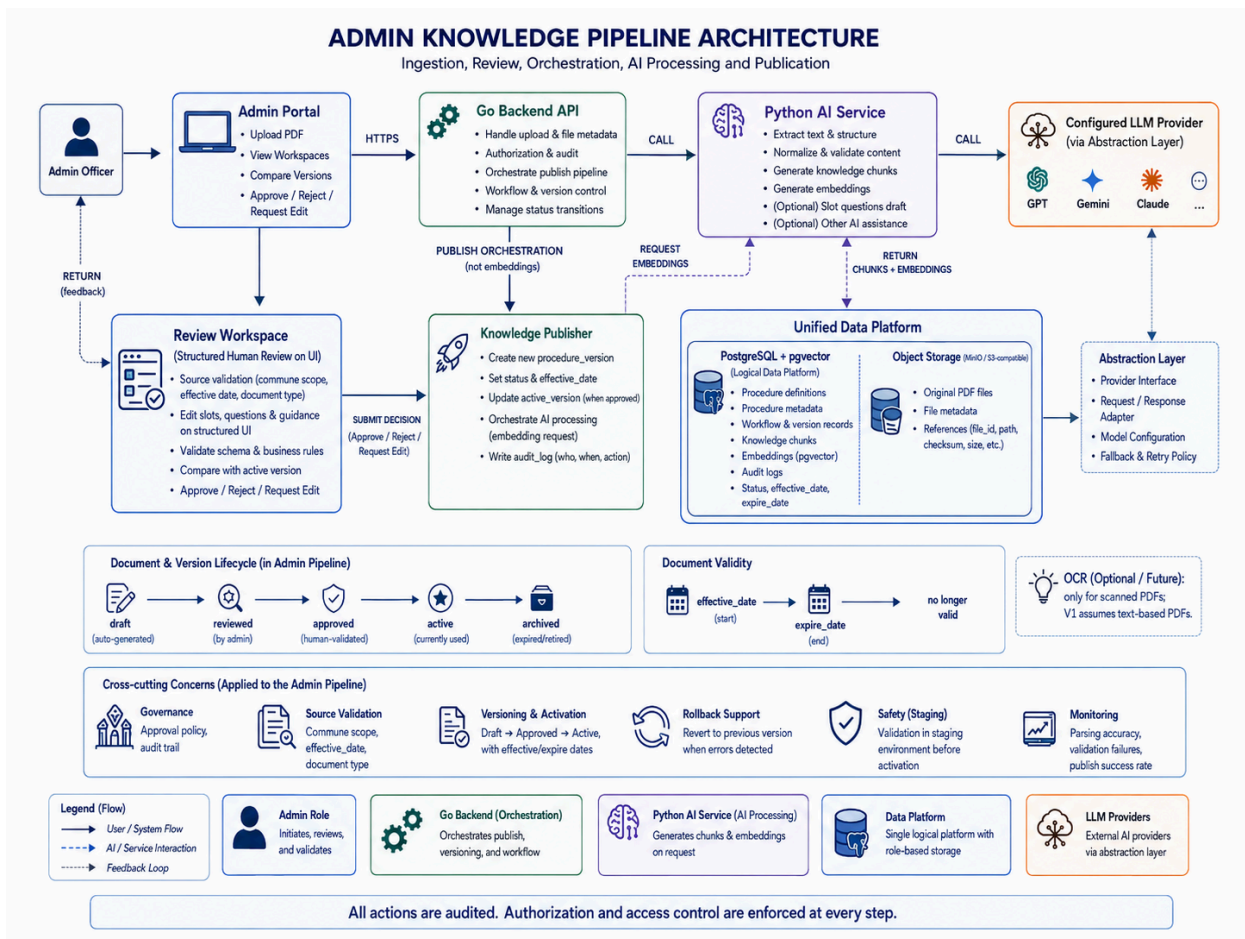


Figure 3.1.2 Admin Knowledge Pipeline Architecture



3.2 Core component Design and Interfacing

3.2.1 Citizen Runtime Components

The citizen runtime is responsible for processing citizen requests, maintaining conversational context, evaluating procedural requirements, and generating grounded administrative guidance. Its design separates deterministic workflow execution from probabilistic AI processing so that the system remains predictable, testable, and auditable.

Citizen Portal

The Citizen Portal provides the primary conversational interface for end users. It allows citizens to submit natural-language questions, review conversation history, and provide additional information when requested by the system.

The portal communicates only with the Go Backend API through secured HTTPS endpoints. It does not access the database, object storage, Python AI Service, or external LLM provider directly. Its responsibility is limited to presentation and user interaction, while authentication, workflow execution, session management, procedural validation, and AI processing remain under backend control.

Authentication Service

The Authentication Service verifies JSON Web Tokens, loads user profiles, and determines the user's access permissions. For authenticated citizens, it associates conversation sessions with a persistent user account. The prototype may also support guest sessions for public guidance, while all administrative functions require authenticated role-based access.

Identity data is logically separated from application data while remaining within the same PostgreSQL deployment in the initial version. This approach preserves clear data boundaries without introducing the operational overhead of maintaining multiple database instances.

Conversation Manager

The Conversation Manager maintains the active conversation session and coordinates message-level processing. Its main responsibilities include:

- Creating or retrieving the current conversation session.
- Persisting incoming and outgoing messages.
- Maintaining conversational context across multiple turns.
- Requesting intent, domain, and candidate slot extraction from the Python AI Service.
- Forwarding extracted structured information to the Procedure Orchestrator.
- Returning the selected response to the Citizen Portal.

The Conversation Manager may use AI-generated results to interpret the citizen's message, but

it does not determine the final workflow action. The final route is always selected by the Decision Policy Engine based on the active procedure definition and the current `slot_state`, rather than by the LLM.

Procedure Orchestrator

The Procedure Orchestrator represents the deterministic business core of the citizen runtime. It loads the currently active procedure definition based on the identified domain and intent, then evaluates the collected citizen information against the procedure's required and conditional slots.

A `slot_state` is maintained for each procedure within a conversation session. Each slot may have one of the following states:

- `missing`: the required information has not been provided.
- `known`: a candidate value has been extracted from the conversation.
- `confirmed`: the value has been explicitly confirmed or accepted.

The Procedure Orchestrator is responsible for:

- Loading the active procedure version.
- Reading the required and conditional slot rules.
- Validating extracted slot values.
- Updating the session-specific `slot_state`.
- Computing the complete set of missing slots.
- Loading predefined questions for all missing slots from the procedure definition.
- Providing workflow evaluation results to the Decision Policy Engine.

When several slots are missing, the orchestrator groups their predefined questions into a single follow-up message whenever practical. The system therefore requests all currently missing information together instead of enforcing a rigid one-question-at-a-time interaction.

The question wording is taken from the approved `procedure_definition`, rather than freely generated by the LLM. This ensures that the information requested from citizens remains consistent with the published procedure version.

Decision Policy Engine

The Decision Policy Engine converts the deterministic evaluation result into a controlled next action. It selects one of three predefined routes:

- `direct_answer`

- ask_all_missing_slots
- provide_final_guidance

The `direct_answer` route is selected when the citizen's first message already contains sufficient information or when the identified request does not require additional procedural slots.

The `ask_all_missing_slots` route is selected when one or more required or conditionally required values remain unavailable. In this case, the system requests all currently missing information in a consolidated follow-up message using questions defined in the active procedure version.

The `provide_final_guidance` route is typically selected after one or more follow-up turns, once previously missing mandatory information has been collected, validated, and confirmed.

This distinction ensures that the LLM does not control conversation routing. Every route is selected through deterministic rules and can be recorded together with the procedure version, slot state, and decision result used at that point in the conversation.

Knowledge Service / RAG Orchestrator

The Knowledge Service operates within the Go Backend as an orchestration component for Retrieval-Augmented Generation. It coordinates approved procedural data, retrieval requests, citations, and final response generation, but it does not directly generate embeddings or invoke provider-specific LLM APIs.

Its responsibilities include:

- Loading the active procedure definition and approved document references.
- Requesting semantic retrieval from the Python AI Service.
- Supplying the procedure version, citizen context, and workflow result.
- Receiving relevant knowledge chunks and source citations.
- Requesting a grounded natural-language response.
- Returning the final response to the Conversation Manager.

Authoritative procedural elements are obtained from the active `procedure_definition`. These include:

- Required documents and document checklists.
- Eligibility conditions.
- Required and conditional slots.
- Submission locations.
- Fees, where applicable.

- Mandatory procedural steps.
- Follow-up questions.
- Workflow and decision rules.

RAG supplements this structured guidance with relevant regulatory context, explanations, and supporting citations. Retrieved content may enrich the final answer, but it must not replace, modify, or override the approved rules stored in the procedure definition.

Therefore, the structured procedure JSON controls the workflow and official checklist, while RAG improves explanation quality and source traceability.

Python AI Service

The Python AI Service performs probabilistic and language-oriented tasks that are unsuitable for the deterministic business layer. Its citizen-runtime responsibilities include:

- Intent and administrative domain detection.
- Structured slot extraction from citizen messages.
- Embedding generation.
- Semantic retrieval through pgvector.
- RAG context construction.
- Grounded natural-language response generation.
- Output validation and safety filtering.

The service communicates with a configured external LLM provider through a provider abstraction layer. One provider is selected at runtime, but provider-specific authentication, request formats, response formats, retry rules, and error handling remain encapsulated inside the Python AI Service.

The Go Backend, Procedure Orchestrator, Decision Policy Engine, and Citizen Portal remain unaware of whether the active implementation uses GPT, Gemini, Claude, or another compatible provider.

Runtime Data Persistence

The citizen runtime uses PostgreSQL with pgvector as a unified logical data platform. Identity data and application data may be shown as separate logical schemas in the architecture diagram, but they remain within the same PostgreSQL deployment in the initial version.

The runtime stores:

- Users, roles, and profiles.
- Conversation sessions.

- Conversation messages.
- Session-specific `slot_state`.
- Procedure definitions.
- Procedure versions and active-version references.
- Document metadata.
- Knowledge chunks.
- Vector embeddings.
- Routing decisions.
- Version references used for each response.
- Audit logs.

During normal citizen interactions, the system primarily reads active procedure definitions, indexed knowledge chunks, embeddings, document metadata, and citation references from PostgreSQL and pgvector.

Original administrative PDF files remain in S3-compatible object storage. They are accessed only when a source document must be displayed, downloaded, verified, or reprocessed. The system does not reopen or parse the original PDF for every conversation request.

OCR remains an optional future capability for scanned documents. The initial version assumes that administrative PDFs contain an accessible text layer.

Citizen Runtime Interaction Flow

A typical citizen request is processed as follows:

1. The citizen submits a message through the Citizen Portal.
2. The Go Backend verifies the session and loads the current conversation context.
3. The Conversation Manager sends the message to the Python AI Service for intent, domain, and candidate slot extraction.
4. The Procedure Orchestrator loads the active procedure definition and updates the current `slot_state`.
5. The Procedure Orchestrator computes all missing required and conditional slots.
6. The Decision Policy Engine selects `direct_answer`, `ask_all_missing_slots`, or `provide_final_guidance`.
7. When information is missing, the system loads all relevant questions from the procedure definition and returns them in one consolidated message whenever practical.
8. When guidance can be provided, the Knowledge Service loads structured procedure data and requests supporting retrieval from the Python AI Service.
9. The Python AI Service retrieves relevant knowledge chunks and generates a grounded answer with citations.

10. The response, selected route, slot state, procedure version, and citation references are persisted for auditability.
11. The final response is returned to the Citizen Portal.

This component design ensures that AI supports language understanding, retrieval, and response formulation, while deterministic backend components retain control over procedural rules, missing-information evaluation, and conversation routing.

3.2.2 Admin Knowledge Pipeline Components

The admin knowledge pipeline transforms official administrative documents into structured, reviewed, version-controlled knowledge. AI supports extraction and drafting, but no generated content is available to citizens until it has been validated and approved by an authorized officer.

Admin Portal

The Admin Portal allows authorized officers to:

- Upload official PDF documents and metadata.
- Review AI-generated procedure drafts.
- Edit slots, questions, conditions, checklists, guidance, and citations through structured forms.
- Approve, reject, return for revision, publish, compare versions, and initiate rollback.

The portal never writes directly to PostgreSQL or pgvector. All operations are submitted through the Go Backend API.

Authentication and Authorization

Administrative functions require authenticated role-based access. Roles may include editor, reviewer, approver, and administrator.

For the V1 prototype, these responsibilities may be collapsed into fewer roles; however, publication and rollback remain privileged actions.

All significant actions are associated with the authenticated user and recorded in the audit log.

Document Ingestion

The Go Backend validates the upload request, stores the original PDF in S3-compatible object storage, and creates a document record in PostgreSQL.

Stored metadata includes:

- Title, document number, and issuing authority.
- Administrative domain and commune scope.
- Effective date and expiry date.
- Document type, checksum, file location, and processing status.

The initial version processes text-based PDFs. OCR is optional and reserved for future support of scanned documents.

Knowledge Processing Service

The Python AI Service extracts and normalizes document text, identifies candidate procedures, and generates structured drafts.

It may extract:

- Required documents and eligibility conditions.
- Fees, locations, processing steps, and processing time.
- Required and conditional slots.
- Questions for missing slots.
- Guidance and source references.

The generated result remains a draft and cannot be used by the citizen runtime before approval.

Procedure Draft Storage

AI-generated and manually edited drafts are stored in `procedure_drafts`, or an equivalent draft repository.

These records are separate from approved `procedure_versions`:

`Procedure_drafts` → editable and not citizen-facing

`Procedure_versions` → approved, immutable, and version-controlled

`Active_version` → currently used by the citizen runtime

A typical draft lifecycle is:

Draft → Reviewed → Approved

A rejected draft cannot proceed to publication. A draft returned for revision remains editable and re-enters the review process after correction.

Review Workspace

The Review Workspace provides structured forms rather than raw JSON editing. Authorized officers can:

- Compare the draft with the source document.
- Validate commune scope, issuing authority, and document type.
- Verify effective and expiry dates.
- Edit slots, questions, conditions, checklists, fees, locations, and guidance.
- Validate schema, business rules, and source citations.
- Preview the expected citizen conversation flow.
- Approve, reject, or return the draft for revision.

Source validation is performed in this workspace because legal scope and document validity require explicit human confirmation.

Validation

Before approval or publication, the backend validates that:

- Required fields and slot identifiers are valid.
- Conditional rules reference supported slots.
- Questions are linked to valid slot definitions.
- Required-document lists and submission locations are complete where applicable.
- Effective and expiry dates are logically consistent.
- The source document applies to the intended commune.
- Expired or invalid documents cannot support a new active procedure.

Validation failures return the draft to the Review Workspace for correction.

Knowledge Publisher

The Knowledge Publisher operates within the Go Backend. It:

- Verifies publication permissions.
- Confirms that the draft is approved.
- Creates an immutable `procedure_version`.
- Links the version to supporting documents.
- Requests chunking and embedding generation from the Python AI Service.
- Stores the resulting chunks and vectors in PostgreSQL with pgvector.
- Activates the new version transactionally.
- Archives the previous version where appropriate.
- Records the publication event in the audit log.

The Go Backend coordinates publication but does not generate embeddings directly.

Embedding and Indexing

After approval:

1. The Admin Portal submits a publish request to the Go Backend.
2. The Knowledge Publisher validates and creates the new procedure version.
3. The Go Backend requests chunking and embedding generation from the Python AI Service.
4. A configured multilingual or Vietnamese-capable embedding model generates vector representations.
5. The AI Service returns chunks, embeddings, metadata, and source references.
6. The Go Backend stores them in PostgreSQL with pgvector.
7. The new procedure version becomes active only after indexing succeeds.

This prevents the citizen runtime from using a version with an incomplete retrieval index.

Procedure and Document Lifecycle

A procedure version may follow:

draft → reviewed → approved → active → archived

A source document may follow:

uploaded → processed → validated → effective → expired or superseded

Document `effective_date` and `expiry_date` determine whether it may support an active procedure version. Historical procedure versions remain available for traceability and audit.

Rollback

Rollback restores a previous approved version without deleting the current version. It requires authorization, validation of the supporting document, a rollback reason, and a complete audit record.

The active-version reference is changed while all previous versions remain immutable.

Data Persistence

The admin pipeline uses the same PostgreSQL and pgvector deployment as the citizen runtime.

It stores:

- Documents and metadata.

- `procedure_drafts`.
- `procedure_versions`.
- Active-version references.
- Knowledge chunks and embeddings.
- Review and approval status.
- Source relationships and validity dates.
- Audit logs.

Original PDFs remain in object storage. Draft, rejected, and incomplete procedures are excluded from citizen-facing retrieval.

Admin Knowledge Pipeline Flow

1. An officer uploads a text-based PDF and metadata.
2. The Go Backend stores the PDF in object storage and metadata in PostgreSQL.
3. The Python AI Service extracts content and creates a structured draft.
4. The draft is stored in `procedure_drafts`.
5. The officer reviews and corrects the draft through the Review Workspace.
6. The backend performs schema, business, source, and validity checks.
7. The officer approves the draft.
8. The Admin Portal sends the publish command to the Go Backend.
9. The Knowledge Publisher creates a new `procedure_version`.
10. The Python AI Service generates chunks and embeddings.
11. PostgreSQL and pgvector store the version and retrieval data.
12. The new version becomes active only after successful publication and indexing.
13. The previous version is archived where applicable.
14. The complete operation is recorded in the audit log.

This pipeline combines AI-assisted extraction, human approval, deterministic validation, versioned publication, and controlled activation.

3.2.3 Data and Storage Components

The proposed system uses PostgreSQL with pgvector as the main data platform and S3-compatible object storage for original documents.

PostgreSQL

PostgreSQL stores structured application data, including:

- Users and roles

- Conversation sessions and messages
- Session-specific `slot_state`
- Documents and metadata
- `procedure_drafts`
- `procedure_versions`
- Active-version references
- Review and approval status
- Audit logs

Identity data and application data may be separated logically by schema, while remaining in the same PostgreSQL deployment for V1.

pgvector

The `pgvector` extension stores embeddings for approved knowledge chunks. It supports semantic retrieval for RAG while keeping relational data, document metadata, procedure versions, and vector data within the same database platform.

Only approved and active knowledge is available to citizen-facing retrieval. Draft, rejected, or incomplete content is excluded.

Object Storage

Original PDF files are stored in S3-compatible object storage, such as MinIO. PostgreSQL stores only file metadata, checksums, document validity dates, and storage references.

The citizen runtime normally reads indexed knowledge and citation metadata rather than reopening the original PDF. Source files are accessed mainly for review, download, verification, or reprocessing.

Data Relationships

Each active procedure references one approved `procedure_version`. A procedure version may reference one or more supporting documents, while each knowledge chunk links back to its source document and procedure version.

This relationship allows every citizen response to be traced to:

- The procedure version used
- The supporting document
- The retrieved knowledge chunks
- The selected citations

- The effective and expiry dates

Data Integrity

The system uses foreign keys, transactions, status controls, and immutable version records to protect data consistency.

Publication updates the procedure version, retrieval index, and active-version reference as one controlled operation. If indexing fails, the new version is not activated.

This storage design supports structured workflow execution, semantic retrieval, document traceability, version management, and auditability without introducing multiple database platforms.

3.2.4 Inter-service Communication

The proposed system uses RESTful APIs over HTTPS for communication between the frontend, Go Backend, and Python AI Service.

Frontend to Go Backend

The Citizen Portal and Admin Portal communicate only with the Go Backend. The backend handles authentication, validation, workflow execution, data access, and response delivery.

Example operations include:

- Sending citizen messages
- Loading conversation history
- Uploading administrative documents
- Reviewing and approving drafts
- Publishing or rolling back procedure versions

Go Backend to Python AI Service

The Go Backend invokes the Python AI Service for AI-specific tasks, including:

- Intent and slot extraction
- Document processing
- Draft generation
- Semantic retrieval
- Embedding generation
- Grounded response generation

The backend sends structured requests containing identifiers, procedure context, conversation context, and authorized document references. The AI Service returns structured JSON responses rather than directly modifying business data.

AI Service to LLM Provider

The Python AI Service communicates with the configured LLM provider through a provider abstraction layer. Provider-specific authentication, request formats, and response handling remain isolated inside the AI Service.

Data Access

The Go Backend is responsible for transactional business data and publication state. The Python AI Service accesses vector retrieval or embedding functions only through controlled service operations.

Original PDF files are accessed through authorized object-storage references. Public clients do not access internal storage services directly.

Reliability

All inter-service calls use:

- Request timeouts
- Retry rules for temporary failures
- Structured error responses
- Correlation IDs
- Logging and audit metadata

AI failures must not corrupt workflow state. If an AI request fails, the backend preserves the current session or draft state and returns a controlled error or retry response.

This communication model keeps business control within the Go Backend while allowing the Python AI Service to evolve independently.

3.2.5 Security, Auditability, and Failure Handling

The system applies authentication, authorization, validation, logging, and controlled failure handling across both citizen and administrative workflows.

Security

Citizen and admin requests are protected through HTTPS and JWT-based authentication.

Role-based authorization restricts sensitive operations, especially document approval, publication, and rollback.

Database credentials, object-storage credentials, and LLM API keys are stored in environment-specific secret configurations rather than source code. Public clients cannot access PostgreSQL, pgvector, object storage, or the Python AI Service directly.

Validation and Guardrails

All API requests are validated before processing. Administrative drafts must pass schema, business-rule, source, and validity checks before publication.

AI outputs are treated as untrusted intermediate results. Structured responses are validated before they are accepted, and retrieved content cannot override the approved `procedure_definition`.

Auditability

The system records significant events, including:

- Conversation routes and `slot_state` changes
- Procedure version used for each response
- Retrieved citations and source documents
- Draft review and approval actions
- Publication and rollback operations
- Authenticated user and timestamp

Correlation IDs and structured logs support tracing requests across the Go Backend and Python AI Service.

Failure Handling

Inter-service calls use timeouts, limited retries, and structured error responses. Temporary AI or LLM failures do not delete the current conversation state, procedure draft, or approved version.

If document processing or embedding generation fails, the draft remains unpublished. A new procedure version becomes active only after validation, persistence, and indexing are completed successfully.

This design ensures that sensitive operations remain controlled, AI outputs remain governed, and system failures do not corrupt workflow or knowledge state.

3.3 Software Engineering Methodology and DevOps Pipeline

Development Methodology

The project follows an iterative Agile approach. The system is divided into small functional increments, including citizen conversation, procedure workflow, document processing, admin review, publication, and RAG integration. Each iteration includes design, implementation, testing, and review.

Version Control

Source code is managed using Git with separate branches for development and stable releases. Changes are submitted through pull requests and reviewed before being merged into the main branch.

Testing Strategy

The project applies:

- Unit tests for workflow rules, validation, and service logic.
- Integration tests for Go, Python, PostgreSQL, and object storage.
- API tests for citizen and admin workflows.
- End-to-end tests for conversation, review, publication, and rollback flows.
- AI evaluation for intent extraction, slot extraction, retrieval relevance, and grounded responses.

CI/CD Pipeline

The CI/CD pipeline automatically performs:

1. Source code checkout.
2. Linting and static analysis.
3. Unit and integration tests.
4. Container image build.
5. Security and dependency checks.
6. Deployment to the test environment.
7. Manual approval before production deployment.

Deployment

The React frontend, Go Backend, and Python AI Service are packaged as separate Docker containers. Docker Compose is used for V1 deployment because the system has limited services and commune-level workload.

Monitoring

Structured logs, correlation IDs, health checks, and basic performance metrics are used to monitor service availability, errors, response time, and AI request failures.

This methodology supports controlled development, repeatable deployment, early defect detection, and maintainable system delivery.

4. EXPECTED RESULTS AND PERFORMANCE EVALUATION

4.1 Deliverables

Software Deliverables

The project will deliver a working citizen assistance system consisting of:

- Citizen Portal for conversational administrative guidance.
- Admin Portal for document upload, review, approval, publication, and rollback.
- Go Backend for authentication, sessions, workflow execution, and knowledge management.
- Python AI Service for document processing, extraction, RAG, embeddings, and response generation.
- PostgreSQL with pgvector and S3-compatible object storage.

Technical Deliverables

The project will also provide:

- Source code repository with setup instructions.
- Docker Compose deployment configuration.
- Database schema and migration scripts.
- REST API documentation using Swagger/OpenAPI.
- Procedure definition schema and sample administrative procedures.
- Automated test cases and evaluation reports.

Documentation Deliverables

The final documentation includes:

- System architecture and database design.
- Installation and deployment guide.
- Citizen and admin user manuals.
- Testing and performance evaluation results.
- Known limitations and future improvements.

These deliverables provide both an operational prototype and the technical documentation required for deployment, maintenance, and evaluation.

4.2 Verification and Validation Plan

Detail the automated and manual testing strategies designed to guarantee system reliability, performance, and compliance with requirements.

Some examples:

Testing Type	Methodology and Tools	Success Criteria / Target Metrics
Unit Testing	Test the Decision Engine, slot_state, validation rules, publication logic, and critical Go/Python service functions.	Minimum 80% statement and branch coverage.
Integration Testing	Verify communication among the Go Backend, Python AI Service, PostgreSQL, pgvector, object storage, and LLM integration. Mock LLM responses are used in CI, while live-provider integration is tested in staging.	Zero critical path connection dropouts.
End-to-End Testing	Test complete citizen and admin flows, including conversation, document review, publication, indexing, and rollback.	All critical workflows complete successfully and preserve the correct procedure version and audit data.
AI Evaluation	Evaluate intent detection, slot extraction, retrieval relevance, citation completeness, and unsupported responses using Vietnamese administrative test cases.	Intent accuracy $\geq 85\%$; slot extraction F1 $\geq 80\%$; citations included in 100% of final guidance responses; unsupported-response rate $\leq 5\%$.
Performance Testing	Measure non-AI API latency, retrieval time, AI-assisted chat latency, and concurrent sessions.	Non-AI API P95 < 500 ms; AI-assisted chat turn average < 5–7 seconds, depending on the configured LLM; support at least 20 concurrent sessions under

Testing Type	Methodology and Tools	Success Criteria / Target Metrics
		prototype commune-scale load.
Security Testing	Test authentication, role-based authorization, input validation, file uploads, secrets, dependencies, and common OWASP risks.	No unresolved high or critical security vulnerabilities.
User Acceptance Testing	Citizens and commune officers evaluate usability, answer clarity, procedural accuracy, and administrative operations.	No unresolved critical usability or procedural-accuracy issues before final evaluation.

4.3 Success Metrics and SLA Compliance

Metric Category	Target
System Availability	≥ 99% during the prototype evaluation period.
Non-AI API Performance	P95 response time < 500 ms.
AI-assisted Response Time	Average chat turn < 5–7 seconds, depending on the configured LLM.
Concurrent Usage	At least 20 concurrent citizen sessions under commune-scale prototype load.
Intent Detection	Accuracy ≥ 85%.
Slot Extraction	F1-score ≥ 80%.

Metric Category	Target
Citation Completeness	Citations included in 100% of final guidance responses.
Unsupported Responses	Hallucination or unsupported-answer rate $\leq 5\%$.
Workflow Correctness	100% of critical test cases follow the approved procedure_definition.
Publication Integrity	No active version is published before validation and indexing succeed.
Security	No unresolved high or critical vulnerabilities.
User Acceptance	Average satisfaction score $\geq 4/5$ from pilot users.

5. PROJECT IMPLEMENTATION AND RESEARCH PLAN

5.1 Work Breakdown Structure (WBS) and Project phases

Phase	Main Activities	Expected Output
1. Requirements and Analysis	Confirm scope, supported procedures, user roles, workflows, and non-functional requirements.	Finalized requirements and use cases.
2. Architecture and Data Design	Design system architecture, database schema, procedure definition format, APIs, and security model.	Architecture diagrams, database design, and API specifications.
3. Core Backend Development	Implement authentication, conversation sessions, Decision Engine, procedure	Functional Go Backend and database layer.

Phase	Main Activities	Expected Output
	orchestration, admin functions, and audit logging.	
4. AI and Knowledge Pipeline	Implement document extraction, draft generation, slot extraction, RAG, embeddings, and LLM abstraction.	Functional Python AI Service and knowledge pipeline.
5. Frontend and Integration	Develop Citizen Portal and Admin Portal, then integrate frontend, backend, AI services, storage, and LLM provider.	Integrated end-to-end prototype.
6. Testing and Deployment	Conduct functional, AI, performance, security, and user acceptance testing; deploy using Docker Compose.	Evaluated prototype, deployment package, and final documentation.

5.2 Project timeline and Milestones

Period	Main Activities	Milestone
Weeks 1–2	Finalize requirements, supported procedures, use cases, and evaluation criteria.	Requirements approved.
Weeks 3–4	Complete architecture, database schema, API design, and procedure definition format.	Technical design completed.
Weeks 5–7	Develop Go Backend, authentication, sessions, Decision Engine, and admin	Core backend completed.

Period	Main Activities	Milestone
	APIs.	
Weeks 8–10	Develop Python AI Service, document processing, slot extraction, RAG, and embedding pipeline.	AI pipeline completed.
Weeks 11–12	Develop Citizen Portal and Admin Portal; integrate all services.	Integrated MVP released.
Weeks 13–14	Conduct functional, AI, integration, performance, and security testing.	Full system test completed.
Weeks 15–16	Fix defects, perform user acceptance testing, deploy the prototype, and complete documentation.	Final prototype and report delivered.

5.3 Feasibility assessment and Risk mitigation Strategy

Evaluate operational, technical, resource, and scheduling risks that could jeopardize project success. Provide actionable mitigation strategies for each identified threat.

Risk Category	Identified Risk	Mitigation Strategy
Technical	Integration complexity between Go, Python, PostgreSQL, pgvector, object storage, and the LLM provider.	Build early integration prototypes and define stable REST contracts.
AI Quality	Incorrect slot extraction, weak retrieval, or unsupported responses.	Use structured outputs, approved procedure definitions, citations, test datasets, and human review.

Risk Category	Identified Risk	Mitigation Strategy
Data	Limited or inconsistent administrative documents.	Start with selected procedures and require source, scope, and validity verification.
External Service	LLM API latency, cost, quota, or provider changes.	Use provider abstraction, request limits, retries, mock responses in CI, and configurable providers.
Security	Unauthorized access to admin functions or sensitive credentials.	Apply JWT, role-based access, secret management, validation, and audit logging.
Schedule	Scope growth may delay core implementation.	Prioritize V1 features and defer OCR, voice, and external government integration.
Operational	Limited infrastructure and technical support at commune level.	Use Docker Compose, simple deployment scripts, documentation, and backup procedures.
User Adoption	Citizens or officers may find the system difficult to use.	Conduct early usability testing and refine interfaces based on feedback.

6. REFERENCES

List all industry papers, technical documentation, whitepapers, structural benchmarks, and relevant academic publications consulted during proposal formulation. **Format:** Use the **IEEE reference style** for technical consistency across software engineering artifacts.